FACULTATIF · HORS DES 18 HEURES

Choisir une extension après le cœur

- Les quatre TP obligatoires suffisent au barème complet.
- Régression : comparer à complexité/équipe égales, sous de nouvelles hypothèses.
- Bootstrap : simuler le rééchantillonnage dans chaque groupe.
- MapReduce : combiner sommes et effectifs ; comprendre le passage par clé.

> **PLAN DU COORDINATEUR**
>
> Matrices/apply et objets sont introduits dans le cœur ; approfondissement matriciel, régression et MapReduce deviennent des extensions. Aucun déploiement RHadoop réel n'est implémenté. L'alignement avec le plan officiel doit être validé par le coordinateur.

FACULTATIF · Workflow ajusté sur complexité et équipe

15 · Régression : ajouter des variables, changer la question

$$E(Y \mid W, C, T) = \beta_0 + \beta_W W + \beta_C C + \beta_B I_B + \beta_D I_D$$

Modèle linéaire additif : durée attendue conditionnelle au contexte.

Y est resolution_minutes ; W vaut 1 pour workflow B et 0 pour A ; C est complexity (de 1 à 5) ; I_B vaut 1 pour l'équipe Boreal ; I_D vaut 1 pour Cedre. Atlas est l'équipe de référence. E désigne une moyenne attendue. Les coefficients β sont des paramètres estimés ; β_W est une différence ajustée en minutes, β_C des minutes par point de complexité. Team et complexity sont mesurées avant le workflow.

1 $$\hat{Y} = 20 - 3W + 4C + 2I_B - I_D$$

Exemple pédagogique de coefficients inventés, pas des résultats Noria.

2 $$C = 2,\ T = \text{Atlas} : \quad \hat{Y}_A = 28, \quad \hat{Y}_B = 25$$

Même complexité et même équipe : prédictions 28 puis 25 minutes.

3 $$\hat{Y}_B - \hat{Y}_A = -3$$

La différence ajustée du modèle additif vaut −3 minutes.

```
# Vérification du petit exemple de prédiction
predire <- function(W, C, IB = 0, ID = 0) {
  20 - 3*W + 4*C + 2*IB - ID
}
predire(1, 2) - predire(0, 2) # -3

# Extension réelle Noria : coefficients à estimer
d <- clean
d$workflow <- factor(d$workflow, levels = c("A", "B"))
d$team <- factor(d$team, levels = c("Atlas", "Boreal", "Cedre"))
fit <- lm(resolution_minutes ~ workflow + complexity + team, data = d)
coef(fit)
confint(fit)
```

CONTINUATION 2 · FACULTATIF · Workflow ajusté sur complexité et équipe

15 · Régression : ajouter des variables, changer la question · suite

LIMITES

L’ajustement répond à une autre question que Welch. Inspecter les résidus et la forme du lien avec complexity ; le modèle impose ici un effet linéaire par point et un contraste de workflow identique dans toutes les équipes. Ajuster seulement des variables pertinentes mesurées avant l’exposition. Un coefficient ajusté ne suffit pas à prouver un effet causal.

FACULTATIF · Une autre construction de l’intervalle

16 · Bootstrap : rééchantillonner les tickets de chaque groupe

$$\hat{\Delta}^{*(b)} = \bar{x}_B^{*(b)} - \bar{x}_A^{*(b)}$$

À chaque répétition b, tirer avec remise dans A et dans B séparément.

$$IC_{\text{percentile}} = [Q_{0.025}(\hat{\Delta}^{*}),\ Q_{0.975}(\hat{\Delta}^{*})]$$

Prendre les quantiles de la distribution des différences rééchantillonnées.

L’étoile indique un rééchantillonnage ; b est son numéro ; on garde n_A et n_B tickets par tirage. Q_p est le quantile de proportion p parmi les répétitions ; la différence et les bornes sont en minutes. Ce bootstrap stratifié suppose ici des tickets indépendants à l’intérieur des groupes.

1 $$A^{*} = (10, 10, 14), \quad B^{*} = (8, 12, 12)$$

Une répétition possible : avec remise, une même valeur peut réapparaître.

2 $$\bar{x}_A^{*} = 34/3, \quad \bar{x}_B^{*} = 32/3$$

Les nouvelles moyennes sont calculées comme dans le tronc commun.

3 $$\hat{\Delta}^{*} = 32/3 - 34/3 = -2/3$$

Cette répétition produit une différence d’environ −0,667 minute.

```
A <- c(10, 12, 14); B <- c(8, 10, 12)
mean(c(8, 12, 12)) - mean(c(10, 10, 14)) # -2/3
set.seed(2026)
R <- 5000
delta_star <- replicate(R, {
  Bs <- sample(B, length(B), replace = TRUE)
  As <- sample(A, length(A), replace = TRUE)
  mean(Bs) - mean(As)
})
quantile(delta_star, c(.025, .975), type = 7)
```

CONTINUATION 2 · FACULTATIF · Une autre construction de l’intervalle

16 · Bootstrap : rééchantillonner les tickets de chaque groupe · suite

LIMITES

Le jouet rend la mécanique visible ; trois valeurs par groupe ne permettent pas d’évaluer correctement la fiabilité d’un bootstrap. Le bootstrap n’invente pas les données manquantes et ne répare ni biais de sélection ni dépendance. Son IC percentile n’est pas l’IC de Welch et peut différer ; garder la graine et le nombre de répétitions.

FACULTATIF · Pourquoi la moyenne des moyennes peut être fausse

17 · MapReduce : transporter somme et effectif

$$\bar{x} = \frac{\sum_{k=1}^{K} S_k}{\sum_{k=1}^{K} n_k}, \qquad S_k = \sum_{i=1}^{n_k} x_{ki}$$

Chaque morceau produit une somme et un effectif ; on les additionne avant de diviser.

K est le nombre de morceaux ; k leur numéro ; n_k est le nombre de valeurs du morceau k ; x_ki est sa ie valeur, en minutes ; S_k est sa somme, en minutes. Les deux signes Σ signifient additionner, soit les valeurs d'un morceau, soit les résumés des K morceaux. La moyenne finale est en minutes.

Morceau	Valeurs (min)	Somme S_k	Effectif n_k	Moyenne
1	10, 20	30	2	15
2	30	30	1	30

1 $$S_1 + S_2 = 30 + 30 = 60, \quad n_1 + n_2 = 2 + 1 = 3$$

Réduction : additionner les résumés de tous les morceaux.

2 $$\bar{x} = 60/3 = 20$$

La vraie moyenne des trois durées est 20 minutes.

3 $$(15 + 30)/2 = 22.5 \neq 20$$

La moyenne non pondérée des moyennes est fausse car les tailles diffèrent.

```
morceaux <- list(c(10, 20), c(30))
# Map : résumer chaque morceau
sommes <- vapply(morceaux, sum, numeric(1))
effectifs <- vapply(morceaux, length, integer(1))
# Reduce : fusionner les résumés
sum(sommes) / sum(effectifs)            # 20
mean(vapply(morceaux, mean, numeric(1))) # 22.5 : piège
```

CONTINUATION 2 · FACULTATIF · Pourquoi la moyenne des moyennes peut être fausse

17 · MapReduce : transporter somme et effectif · suite

LIMITES

Ce code montre l'idée ; il n'exécute aucun calcul distribué. Garder les mêmes règles de validité dans chaque morceau. Des moyennes seules ne suffisent pas à reconstruire une médiane ; une variance demande davantage de résumés ou une méthode de fusion dédiée.

EXEMPLE GUIDÉ · DEUX LOTS, DEUX GROUPES

MapReduce : voir aussi les clés

$$\bar{x}_g = \frac{\sum_k S_{kg}}{\sum_k n_{kg}}$$

S_kg est la somme du groupe g dans le lot k ; n_kg est son effectif.

```
d <- data.frame(g = c("A", "B", "A", "B"),
                x = c(10, 20, 30, 40))
lots <- list(d[1, ], d[2:4, ])
map <- lapply(lots, function(lot) {
  aggregate(cbind(somme = lot$x, n = rep(1, nrow(lot))),
            by = list(g = lot$g), FUN = sum)
})
emis <- do.call(rbind, map)
reduce <- aggregate(cbind(somme, n) ~ g, data = emis, sum)
reduce$moyenne <- reduce$somme / reduce$n
reduce
```

Clé	Somme / n	Moyenne
A	40 / 2	20
B	60 / 2	30

LIMITE

Tout se passe dans un processus local. Les objets intermédiaires et leur structure sont visibles dans examples/mini_mapreduce.R ; cela ne démontre ni réseau ni tolérance aux pannes.

RÉFÉRENCES · SANS ÉVALUATION OBLIGATOIRE

Approfondir à son rythme

Matrices et fonctions : ancien TP approfondi

MapReduce : ancien TP approfondi

Données réelles : airquality

Les solutions de la première édition restent dans legacy/ pour les personnes qui choisissent ces défis. scripts/01_run_all.R reproduit l'analyse complète de référence, y compris modèle ajusté et agrégation ; il n'est pas un contrôle de votre travail étudiant.